

總覽：check 是 depot 裡的 script，機器自動跑；AI agent 寫 script、看結果

這份文件回答：有人 submit 之後，機器做了哪些事；Jenkins 和 trigger 是什麼；AI agent（啟動包要做的那個程式）、CAD、PM 各做哪一段



每一步都是 depot 裡的 script；Jenkins 或 cron 只是按時按它。agent 交 shelved CL 之前自己跑過一次，那就是這條 pipeline 的第一次執行。

AI agent 做的（從自己的機器，用 API） 寫 run_sanity、make_manifest、make_release 寫 Jenkinsfile 與 trigger，交 shelved CL 用 API 建 job、按一次 build、讀 log 與結果 不登進任何 server，不裝 trigger	CAD 做的（各一次） 給 agent 一個限定資料夾的 Jenkins 帳號 告訴 agent 哪個節點有 p4 與 bsub 裝 trigger (super 才能) 不給帳號時：agent 寫好、CAD 貼上建 job	PM 做的 去談帳號、節點、權限 (09 #25) 用 agent 的帳號登入一次，產 API token 給它 擋 submit 之前：核准 bypass 與總開關 看 build 頁與一頁摘要，不用看 Jenkins 設定
---	---	--

每個 check 都是 depot 裡的 script；Jenkins 或 cron 只是定時去跑它；p4 trigger 在 submit 前後把它接上。

AI agent 寫 script 與 job 定義、用 API 看結果，不登進 server；人只做兩件事：開帳號、給 token。

後面：分工、CI 三件事、察覺的兩條路、Jenkins 六個概念、trigger 四種事件、check 三級、CD 接著做的三件事、git 的對應、三個坑。

分工：人只開帳號、給 token；script、job、trigger 都由 AI agent 用 API 做

在哪裡	做什麼	agent 和它的關係
agent 的機器	跑 Claude Code；有 p4 client、slack、Jenkins 的 API token	讀 depot、分析、寫 script、試跑一次、交 shelved CL、讀結果、發訊息
p4 server	trigger：submit 時查 CL 說明，或踢一個 job	agent 寫 trigger script 交 shelved CL；CAD 裝（要請示）
Jenkins	編排：submit 後 sync 乾淨 workspace → run_sanity → manifest → release → 通知	agent 寫 Jenkinsfile 與 job 定義，用 API 建、按、讀；不登進它
算力農場（LSF）	真正跑 EDA	pipeline 用 bsub 送，和工程師一樣；agent 自己試跑也走這裡，用它的預算
取用處與 build 頁	交付包、manifest、check 結果	agent 只從這裡讀；工程師與下游也從這裡拿

人只做兩件事

CAD：給 agent 一個限定資料夾（cicd-mentor/）的 Jenkins 帳號，說哪個節點有 p4 與 bsub。

PM：用那個帳號登入一次產 API token，交給 agent 放 config，不進 depot。

五個地方各做自己的事，中間只靠 depot 裡的 script 與固定位置的結果檔接起來；AI agent 不登進任何 server。

CAD 不給帳號：agent 寫好 Jenkinsfile 與 job 定義，CAD 在網頁上貼一次，之後 agent 只要唯讀。

CI：每次 submit 在乾淨 workspace 跑 sanity、記下結果；submit 前跑才擋得住

CI 在機器上是三件事



兩種時機

submit 後跑：只報告級、警告級



main 可能已經壞了；誰弄壞的當天就知道，不用等整合

submit 前跑：擋



共用的 main 隨時可用；代價是使用者按 submit 要等，check 必須快

check 分三級（啟動包叫上線分級）：從後者走到前者

只報告：submit 後跑，結果只寫下來，不吵人

警告：submit 後跑，私訊本人；誤報率低了才升

擋：submit 前跑；PM 與 owner 同意、有 bypass、有總開關才裝

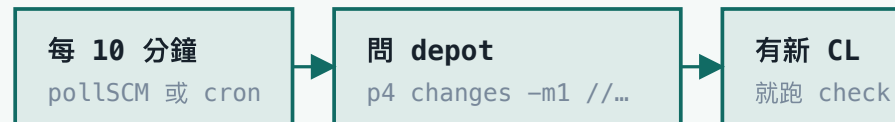
第 6 步只報告，第 9 步才擋 (07-build-brief)

CI = 察覺改動、乾淨環境跑 check、結果寫下來；這三件事 submit 後跑就有價值，submit 前跑才擋得住。

Jenkins 把這一串步驟叫 pipeline；它的前半段就是 CI，後半段才是 CD（第 8 頁）。

察覺改動的兩條路：定時跑 p4 changes，agent 自己能裝；trigger 要 CAD 裝

定時查 (polling)：agent 自己能裝



記下看過的最大 CL 號；重啟不重跑

- ✓ 不碰 p4 server，不用 admin
- ✓ MVP 第 6 步就用這個
- ✗ 最多晚幾分鐘才知道
- ✗ 擋不住：改動已經進去了

有 Jenkins：job 設 pollSCM。沒有：農場節點的 cron 跑一支 script。

trigger：裝在 p4 server 上，CAD 才能裝

p4 server 的一張表 (p4 triggers，super 才能編輯)

每一行：名字 事件 路徑 跑哪支 script
desc-check change-submit //depot/... "desc_check.py %change%"

script 跑在 p4d 那台主機上，拿得到 %change%、%user%、%client%

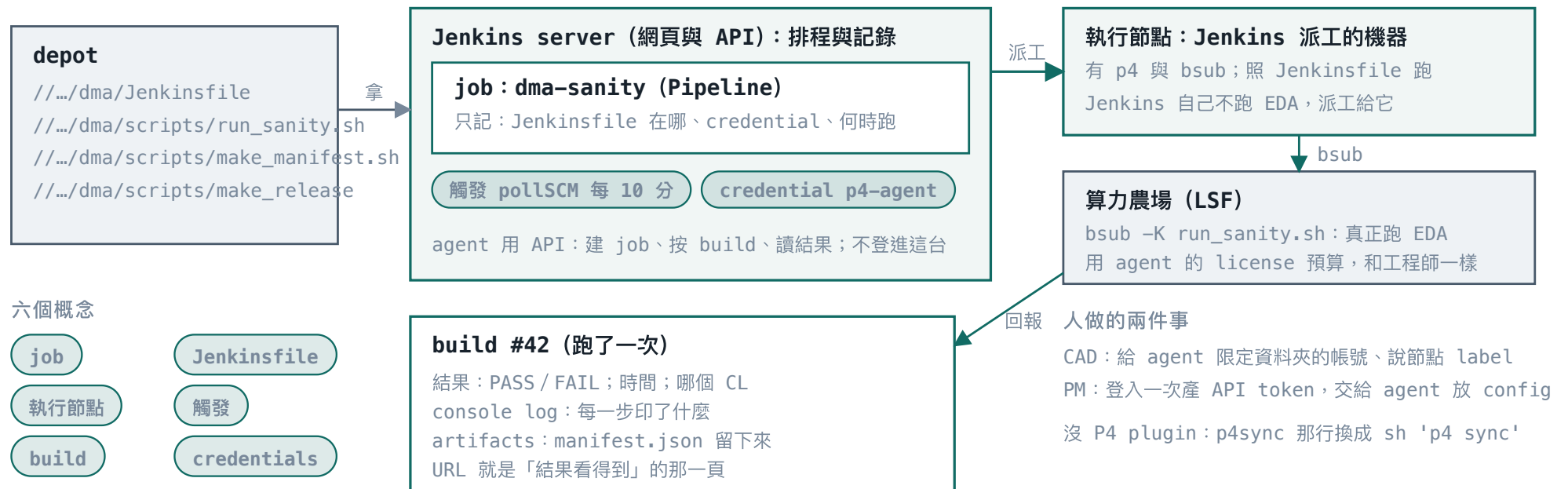
- ✓ submit 當下就跑，能退回
- ✓ shelve 時也能跑，review 看到的是綠的
- ✗ 只有 super 能裝：agent 寫好交 CAD
- ✗ 使用者在等：要快，重的事不能放這裡

agent 先在沙盒的 p4d 裝起來測到對，再交 shelved CL。

察覺改動有兩種方法：定時查 agent 自己就能裝、擋不住；trigger 裝在 p4 server 上、能擋、要 CAD 裝、要快。

先定時查（只報告級），穩定了才談 trigger（第 9 步）。

Jenkins：負責排程與記錄的 server；check 實際在農場跑，每次結果留在 build 頁



Jenkins 只做一件事：在指定的時機，派一台節點照 depot 裡的 Jenkinsfile 跑 (這一串叫 pipeline)，把 log 與產物留在 build 頁。

job 不含任何邏輯，只記去哪拿 Jenkinsfile；所以 agent 用 API 建 job、CAD 給帳號，就接上了。

trigger 兩類：submit 前的能擋、要幾秒跑完；submit 後的叫 Jenkins 跑長 check

一次 submit 在 p4 server 上經過的點



另一條：工程師 shelve → shelve-commit (對 shelved CL 先跑 check，review 的人看到的是已經綠的)

事件	能不能擋	拿來做	要注意
change-submit	能 (exit 非 0 退回)	說明有沒有寫目的；netlist 這類產物有沒有放進 rtl/；一包太大	使用者在等：幾秒內
change-content	能	看內容：filelist 引用的檔在不在、禁止的字串	使用者在等：要快
change-commit	不能	叫 Jenkins 跑 sanity、通知、打 known-good；長的 check 放這裡	已進 depot，壞了只能通知
shelve-commit	不能	對 shelved CL 跑 check；配合「shelve 給人看」的常規	shelve 很頻繁，check 要輕

升級的順序：MVP 第 6 步定時查 (不裝 trigger) → change-commit 叫 Jenkins → 穩定後才裝 change-submit / change-content 擋。

trigger 有四種事件：submit 前的兩種能擋，但使用者在等、幾秒內要跑完；進 depot 後的那種叫 Jenkins 跑長的 check。

agent 寫 script 與那一行定義、在沙盒 p4d 上測好、交 shelved CL；真實 server 由 CAD 裝，裝之前請示。

check 分三級：只報告與警告不擋 submit；擋的才 exit 1，而且要先有 bypass

desc_check.py %change% %user% (示意)

```
desc = p4 -zttag -F %Description% change -o $change
ok = len(desc) >= 20 and not desc.startswith("update")
if not ok:
    print("[cicd-mentor] CL 的說明太短或只寫 update/fix，請寫目的")

sys.exit(0) # 只報告、警告：永遠放行
# 擋：sys.exit(0 if ok else 1)
# 說明裡有 [bypass:原因] 的放行並記錄
# 開頭先讀開關檔：總開關關了就 exit 0
三個等級只差最後幾行；check 的邏輯不變
```

只報告

submit 後跑
永遠 exit 0
結果寫 log、build 頁
出錯：放行

警告

submit 後跑
exit 0，私訊本人
誤報少了才升到擋
出錯：放行

擋

submit 前跑
exit 1 就退回
要同意、bypass、總開關
出錯：預設放行並通知

bypass：CL 說明寫 [bypass:原因]，放行並記錄；PM 看得到誰用了幾次

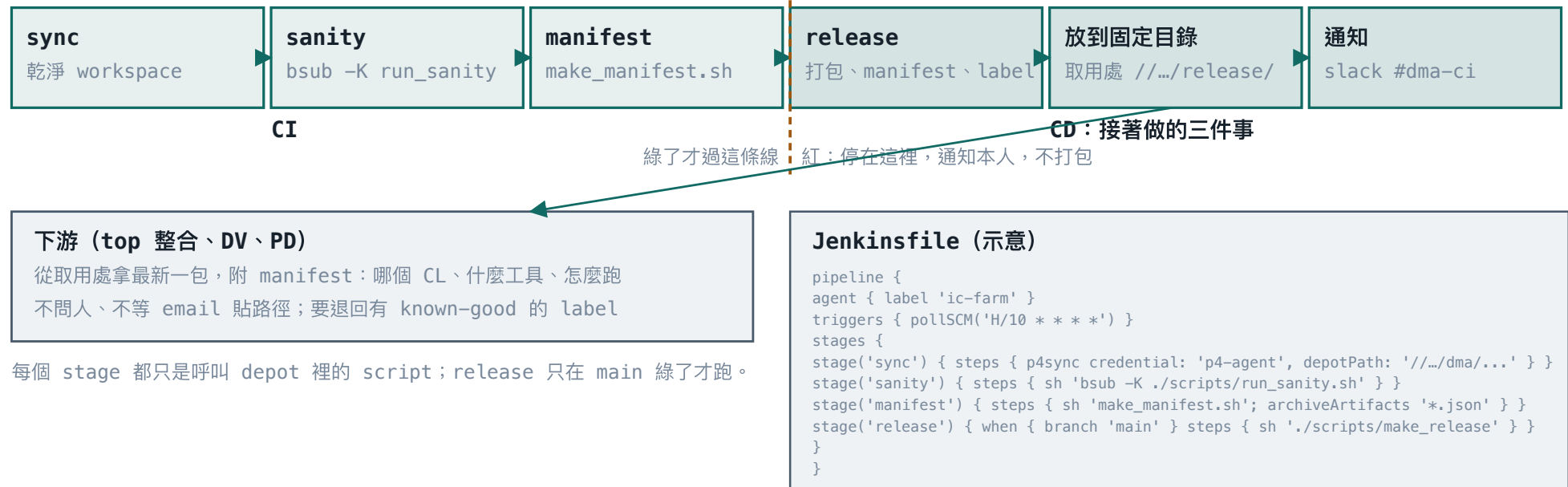
總開關：trigger 表那行註解掉，或 script 開頭讀開關檔；PM 與 admin 都按得到
吵了（誤報、抱怨）就退一級，不硬撐

三級是同一支 script：只報告與警告永遠 exit 0、只寫下來或私訊；擋的才 exit 1，而且 bypass 與總開關要先有。

只報告與警告這兩級一定「出錯也放行」，否則 script 一壞全公司不能 submit。

CD：sanity 綠了之後同一條流程接著自動打包、放到固定目錄、通知下游

同一個 Jenkinsfile 的 stage，由左到右



CD 不是另一套系統：sanity 綠了之後，同一條流程接著做三件事：打包、放到固定目錄（取用處）、通知下游。

AI agent 幫 owner 定出交付物後，把 make_release 寫成 script 交 shelved CL；之後 main 過 check 就自動打包交下游。

用 git 的 project : client 端的 hook 擋不住人硬 push ; 要靠 protected branch 擋

這頁給用 git 的 project ; git 的 hook 兩種，都不是 CI

client 端 : pre-commit、commit-msg、pre-push
在每個人的 .git/hooks，不隨 repo 走、各人自己裝

x 擋不住任何人，只能當提醒

server 端 : pre-receive、post-receive
在 git server 上 ; GitLab / GitHub 不開放自己的 script
→ 改用右邊兩個內建機制

GitLab 內建的兩個機制，就是 CI 與擋

pipeline (自動流程)
.gitlab-ci.yml 在 repo 根目錄
每次 push 與每個 MR 自動跑

runner
跑 job 的機器 = Jenkins 的節點
tag ic-farm : 有 p4 / EDA

protected branch = 擋
master 只能經 MR 併入 ; MR 要 pipeline 綠 + 一人 approve 才能按
不用自己寫 hook ; agent 自己的 repo 走的就是這套

Perforce	git	意思
depot	repo、remote	共用的那份
submit	commit+push	進共用的地方
shelved CL	branch+MR (GitHub 叫 PR)	給人看、還沒進去
stream	branch	一件任務一條線
label	tag	某一刻的檔案清單
change-submit trigger	protected branch+pipeline 必須綠	submit 前擋
change-commit trigger	post-receive、webhook	submit 後踢 CI

用 git 的 project : client 端的 hook 擋不住人硬 push ; pipeline 加 protected branch 才是 CI 與擋。名詞整套切換，原則不變。
agent 自己的 repo 就是這套 : MR、pipeline 跑沙盒三項、master 鎖住。

三個坑：submit 前的 check 太慢、低等級卻擋人、把 client 的 hook 當成 CI

trigger 太慢

症狀

change-submit、change-content
是使用者按下去在等的；
超過幾秒就被罵，然後被要求拆掉

做法

- ✓ submit 前只做秒級的檢查
- ✓ 重的 check 放 change-commit 後
- ✓ 沙盒先量 trigger 跑多久

低等級的 check 壞了卻擋人

症狀

只報告級的 script 出了 exception、
p4 指令失敗、磁碟滿；trigger 回非 0，
所有人的 submit 都被退回

做法

- ✓ 只報告與警告級：出錯也 exit 0
- ✓ 擋的那級：先有總開關與 bypass
- ✓ 出錯時怎麼辦寫進授權表

把 client 端的 hook 當成 CI

症狀

git 團隊裝了 pre-commit 以為有 CI；
沒裝的人照樣推上去

做法

- ✓ 要擋靠 protected branch 或 server 端
- ✓ pre-commit 只當提醒，不當規則
- ✓ Perforce 沒這問題：trigger 在 server

三個坑都來自同一件事：submit 前的 check 有人在等，而且一壞就擋到所有人。

所以先從 submit 後、只報告開始；擋是最後一步，而且要有退路。